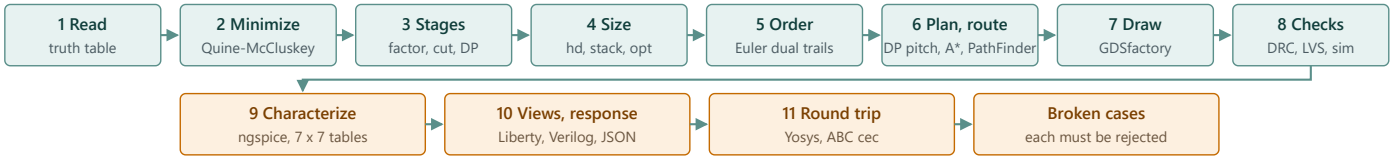
From a Boolean function to a sky130 HD cell: the flow

DD2 Project 1, Option 1, group 5. One request (an expression or a truth table, up to four inputs) becomes a 2.72 um, whole-site cell in the instructor's HD frame, with its schematic, extracted netlists, Liberty and Verilog views, and a JSON response. Each step is a script in `src/` run by `make`; every check writes a JSON report whose `passed` field is the verdict and exits non-zero when it fails or cannot run.



1 Reading the request

`request_reader.py` validates the request (Appendix A format: inputs, output, expression and/or truth table, sizing) and parses the expression by recursive descent over `!`, `&`, `^`, `|` and parentheses, then evaluates it on all 2^n rows. **Input 0 is the least significant bit of the row index**; when both an expression and a table are given they must agree. Every later check compares against this normalized table, never against a cell name. For the six reference cells of section 4.1, `reference_cell.py` takes HD's pin names and transistor sizes from the installed SPICE netlist.

2 Two-level minimization

Quine-McCluskey (`qm.py`): minterms are grouped by their number of ones; implicants that differ in one bit merge into a cube with a don't-care, repeated until nothing merges; the unmerged cubes are the **prime implicants**. Essential primes (the only cover of some minterm) are taken first; **Petrick's method** expands the product of sums over the remaining minterms into all **minimum covers** (fewest terms, then literals). It is run on both f and !f: a cover of f maps to one CMOS structure, a cover of !f to its dual.

3 Stages

`synthesize.py` turns each minimum cover into formula trees by algebraic factoring (common literals pulled out). A static CMOS stage computes the complement of its pull-down network: an AND in the pull-down is a series stack, an OR a parallel one, and the pull-up is the exact dual, leaf for leaf. A **dynamic program** over each tree chooses, per subtree, its polarity and whether to cut it into a stage of its own, either built as its dual ("flip") or followed by an inverter, so that no stack exceeds **three transistors in series**; complemented inputs cost one shared input inverter. The objective is fewest transistors, then fewest inverters, then fewest stages (handout step 2). For functions algebraic factoring cannot reach, a **simple disjoint decomposition** (Ashenhurst: a bound set whose decomposition chart has two distinct columns) gives f = F(h(bound inputs), others); XOR3 becomes (A ^ B) ^ C. The network is checked at switch level (every row: the output conducts only to the correct rail).

4 Sizing: three families

family	output stage	internal stages
hd	every NMOS 0.65 um, PMOS 1.0 um (HD x1)	same
stack	x1 times series depth, folded into fingers that fit the row	x1
opt	x1 (the drive of a _1 cell)	sized for the fastest worst path

opt uses an Elmore RC model fitted to HD's `inv_1` Liberty: gate capacitance per um from its pin capacitance, pull-down and pull-up resistance per um from the slope of `cell_fall`/`cell_rise` against load, self capacitance from the zero-load intercept. A stage's delay is its worst series resistance times the capacitance on its output:

$$d = (n\,R_{unit} / W) \cdot (C_{gate}\Sigma W_{driven} + C_{self}\Sigma W_{drains} + C_L)$$

The worst path from an inv_1-driven input to the nominal load, over every pin and edge, is minimized by coordinate descent over each internal stage's (W_n, W_p), from 0.42 um (the standard-FET width of diff/tap.2) to the row height. Wider internal stages switch faster but load the driver more; the optimum lies between.

5 Transistor ordering (Euler paths)

`euler.py` follows **Maziasz and Hayes**: each network is a graph (nodes = diffusion nets, edges = transistors); a **dual trail** is a sequence of edges that is a trail in both the pull-down and pull-up graphs with the same gate order, so each column is one shared poly gate and neighbours share diffusion. Series stacks may be reordered; an exact memoized search finds the **minimum dual-trail covers** (each extra trail is a diffusion break), ranked by breaks, then repeated nets in a row, then wiring. Folded transistors (stack sizing) use a separate search that keeps a leaf's fingers side by side so **one poly bar** joins them. Stages are ordered and mirrored (permutations, or a beam search) to abut on shared nets; several placements are kept for the planner.

6 Layout planning and routing

`prepare_layout.py` decides every coordinate and writes `layout_plan.json`; drawing never decides geometry. Frame numbers (2.72 um height, 0.46 um site, rails, wells, implants, npc band, L) come only from the instructor's `plan_hd.json`; design-rule values only from `rules.json`, each proven against the installed `sky130A.tech` by `make check`.

- **Gate pitches**: a dynamic program over the gates, left to right, chooses each poly contact's offset and the smallest distance to the previous gate under poly.2, licon.11, pad spacing, li.3 between gate landings and, at a routing channel, one free li1 track. Adjacent gates on one input share a poly bar and one contact.
- **Fixed-landing router**: Manhattan A* on a 0.01 um grid; other nets' shapes are rasterized, inflated by spacing plus half a wire. li1 first; an mcon via to met1 only where li1 cannot route, and only in a multi-stage cell (handout Tables 1 and 3). Up to 12 net orders are tried.
- **Negotiated router** (**PathFinder**): diffusion contacts are not fixed in advance; each may land anywhere along its column with a tall or wide landing. All nets are routed again and again at a cost per grid step of $(1 + \text{history}) \cdot (1 + \text{present} \cdot \text{overlap})$, present rising each iteration and history where nets collided, until no two nets overlap. A quick pass first routes every net alone and skips hopeless attempts.
- **Search order**: every placement, channel set and width, narrowest first, li1 before met1.
- **After routing**: contacts slide under their vias, each contact gets as many licons in parallel as fit, mcons stack over licon columns, same-net notches are filled, and KLayout re-checks spacing and width; every pin keeps a met1 access pad.

7 Drawing

`generate_gds.py` draws the plan with GDSfactory, reusing Tutorial 3's `rect` unchanged, writes the GDS, a preview and the cell's input record. No coordinate is computed here.

8 Checks

check	how
DRC	Magic full DRC (Tutorial 3's script) on the cell in 24 tapped-row contexts beside tap, inv_1 and fill_1 (side by side, mirrored, flipped onto a shared rail); a deliberately broken control must fail
Layer check	nsdm/psdm enclose the diffusion, hvtp covers PMOS, npc encloses every poly contact; copies with a layer removed must be rejected
LVS, RC-LVS	Magic extraction, Netgen against `schematic.spice`; again with resistance extracted and collapsed; reference cells also against the installed HD circuit
Switch level	conduction paths per row, on the network and on the Euler-reordered network
Transistor level	ngspice (BSIM4 stress fix) on the RC netlist: DC on every row, a transient for every single-input edge (settle, no half-supply glitch), all copies in one deck
Model level	Verilog and Liberty function against the truth table (Icarus Verilog)

9 Characterization

`characterize.py` simulates the RC netlist (with SA/SB stress parameters on every transistor) over HD's 7 x 7 slew and load indices. For every input pin, each setting of the other inputs in which that pin controls the output is simulated (rising and falling edges), and per arc the slowest setting is kept, as HD's TT library does: delay from 50 % to 50 %, transition from 20 % to 80 %; input capacitance is the charge the source delivers during its ramp, divided by the supply. Before any generated cell, the characterizer must reproduce the shipped Liberty of the installed reference cells within 5 % (Table 5).

10 Views, response, round trip

`liberty_view.py` writes and checks the cell's Liberty (function, pins, tables, area); `write_response.py` the JSON response and report; all cells are merged into one library. `round_trip.py` places each cell inside `hd_reduced.lib`, maps a one-cell design with the given Yosys/ABC scripts and proves it equivalent to the request with ABC `cec`.

11 How pass and fail are decided

`make check`, `reference`, `all`, `test`, `char`, `views`, `roundtrip` and `test_views` run the steps in order. A check that cannot run fails. Every check has a broken case in `tests/broken/` that it must reject, and `make test` runs them all. Results under `cells/` and `results/` are written only by these scripts.

Details, numbers and the history of each decision: docs/TECHNICAL.md. Cellforge (gui/server.py) walks through these steps for one function in the browser.